UrbanPulse

Real-Time Urban Operations Intelligence & Smart Traffic Management Platform

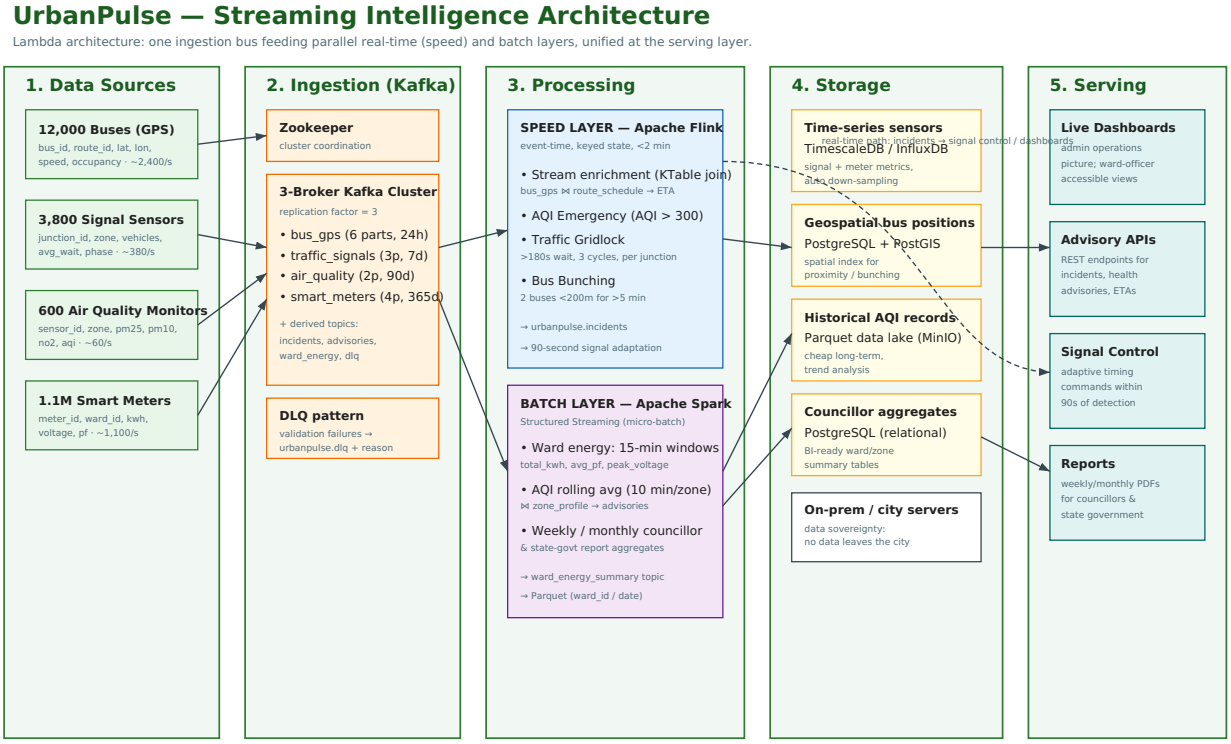
City: MetroConnect (4.2M population) | Smart Cities Mission | Open-source-first mandate
Technical Report — Tasks A, B & C | Lead Streaming Architect submission

> **What this report covers.** Task A — system design (architecture diagram, Lambda vs Kappa evaluation, readiness checklist). Task B — the Kafka ingestion layer (cluster, producers, priority consumers, enrichment, dead-letter queue). Task C — the processing layer (Flink incident detection, Spark ward analytics, and the engine comparison). All referenced code is in the accompanying Git repository; file paths are given throughout.

Task A — Architecture & Design

A.1 Labelled Architecture Diagram

UrbanPulse is a five-layer **Lambda** pipeline: four city data sources feed a 3-broker Kafka ingestion layer; a Flink speed layer handles sub-2-minute incident detection while a Spark batch layer produces auditable aggregates; results land in a purpose-fit storage layer and are surfaced through dashboards, advisory APIs, and the signal-control interface.



Storage technology choice per data class

Data class	Technology	Why this choice
Time-series sensor data (AQI, signals, meters)	**TimescaleDB** (PostgreSQL extension)	Native time partitioning & continuous aggregates; standard SQL for BI tools; open-source and self-hosted on city servers.
Geospatial bus positions	**PostgreSQL + PostGIS**	First-class geospatial types and distance queries ("buses within 200 m") needed for ETA and bunching; open-source.
Historical AQI records	**Parquet on MinIO** (S3-compatible)	Columnar, compressed, cheap long-term retention for 90-day+ pollution trends; queryable by Spark; self-hostable for sovereignty.
Councillor report aggregates	**PostgreSQL**	Small, relational, audited summary tables; easily exported to PDF/Excel for state-government submission.

For the runnable single-laptop demo, the repository uses lightweight equivalents — partitioned Parquet files and Kafka topics — so a beginner can run everything without standing up four databases. The production design specifies the technologies above.

A.2 Lambda vs Kappa — Evaluation Matrix

Each cell is grounded in UrbanPulse's actual requirements.

Criterion	Lambda	Kappa
Latency	Flink speed layer already delivers sub-2-min AQI alerts and 90-s signal adaptation; batch latency applies only to reports, which tolerate it. Meets every SLA.	Single streaming path also meets the sub-2-min SLAs. No latency disadvantage for live use cases.
Fault Tolerance	Two independent paths: a Spark batch failure never blocks real-time incident detection, and vice-versa. Reporting failure cannot silence emergency alerts.	One pipeline is a single point of failure for both alerts and reports; a bad deploy can take down emergency detection and reporting together.
Operational Complexity	Higher: two engines (Flink + Spark), two codebases and failure modes — the real cost of Lambda for a small city team.	Lower: one engine, one mental model, one deployment. Attractive for a lean ops team.
Reprocessing Capability	Batch layer is built for it: re-run a month of Parquet to correct a councillor report after a late sensor recalibration. Natural fit.	Reprocessing = replaying Kafka. The 365-day meter audit would require keeping a year of raw events in Kafka purely to reprocess — expensive and heavy.
Cost	Two clusters to run, but batch can use cheap schedulable capacity and raw history lives in cheap Parquet/object storage, not Kafka.	Must retain very long Kafka history (365-day audit) to preserve reprocessing — high broker-storage cost that dominates at ~3,900 events/s aggregate.
Compliance with Govt Reporting Mandate	Deterministic, re-runnable, auditable aggregates from immutable Parquet — exactly what state-government submission and audit require. Strong fit.	Figures derived from a replayed stream are harder to reproduce identically months later for audit. Weaker compliance story.

Architecture choice: Lambda. UrbanPulse has a genuine dual mandate — hard real-time response *and* long-horizon auditable reporting (including a 365-day smart-meter regulatory audit). Kappa wins on simplicity but loses on the two requirements that matter most here: cheap, auditable, long-horizon reprocessing for compliance, and failure isolation so a reporting bug can never silence an AQI emergency. The 365-day audit retention alone makes Kappa's "replay from Kafka" model prohibitively expensive. We choose **Lambda** — Flink speed layer + Spark batch layer with immutable Parquet as the historical system of record — accepting higher operational complexity as the price of regulatory auditability and fault isolation, and mitigating it by keeping both layers in one Kafka-centric open-source toolchain.

A.3 Architecture Readiness Checklist

16 items across the four mandated themes. A deployment is "ready" only when all are met.

Data Sovereignty

1. All Kafka brokers, processing engines, and databases run on city-owned servers in the municipal data centre; no managed cloud service stores citizen data.
2. No data egress to external networks; MinIO and databases bound to the internal VLAN with egress firewall rules denying outbound traffic.
3. Backups and Parquet snapshots written only to city-controlled storage; off-site DR replica is a second city/state facility, not a public cloud.
4. Access to raw streams is role-restricted; an audit log records who read or exported citizen-linked (meter/ward) data.

Open-Source Mandate

5. Every core component is OSS with an Apache-style licence: Kafka, Zookeeper, Flink, Spark, PostgreSQL/PostGIS, TimescaleDB, MinIO, Parquet, Flask.
6. No proprietary connectors or paid enterprise add-ons on the critical path; any vendor tooling is optional and replaceable.
7. Build/run is reproducible from pinned versions (`requirements.txt` , `docker-compose.yml`); no closed binaries required to operate.

Disaster Recovery — RPO < 15 min, RTO < 30 min

8. Kafka topics use replication factor 3 so a single broker loss causes zero data loss and no downtime.
9. Continuous/near-continuous replication or =<15-min snapshotting of Parquet and databases to the DR site guarantees **RPO < 15 min**.
10. A documented, rehearsed failover runbook (promote DR brokers, repoint clients, restart Flink/Spark from last checkpoint) is provable to complete in **under 30 min (RTO)**.
11. Flink and Spark run with checkpointing to durable storage so stream jobs resume from the last consistent state after failover.
12. DR drills are scheduled (e.g. quarterly) and measured RPO/RTO are recorded against the targets.

Accessibility for Non-Technical Ward Officers

13. Dashboards present plain-language status ("AQI: Hazardous in Zone 3") with colour coding, not raw JSON or query consoles.
14. Advisory and incident views work on a standard browser/mobile with no install and SSO login; no command line is ever required of a ward officer.
15. Dashboards meet basic accessibility: legible fonts, colour-blind-safe palette, regional-language labels.
16. A one-page "what each alert means and who to call" guide ships with the dashboard so non-technical staff can act without engineering help.

Task B — Kafka Ingestion Layer

B.1 3-Broker Cluster, Partitions & Retention

The cluster is defined in `docker-compose.yml`: one Zookeeper plus three Confluent brokers (`kafka1/2/3`) with replication factor 3, reachable on the host at `localhost:9092 / 9093 / 9094`, plus a Kafka-UI on port 8080. Topics are created idempotently by `setup/create_topics.py`.

Topic	Partitions	Retention	Justification
`urbanpulse.bus_gps`	6	24 hours	Highest-volume stream (~2,400/s); 6 partitions spread load and allow parallel consumers. 24 h retention supports accident-replay investigation, per the brief.
`urbanpulse.traffic_signals`	3	7 days	3 partitions map one-to-one to the 3-consumer STANDARD analytics group. A week covers short-term congestion analysis.
`urbanpulse.air_quality`	2	90 days	Low volume (~60/s) needs little parallelism. 90-day retention enables pollution-trend analysis, per the brief.
`urbanpulse.smart_meters`	4	365 days	Keyed by ward for ordered per-ward aggregation. 365-day retention meets the regulatory energy-audit requirement.

Derived topics (`bus_gps_enriched`, `incidents`, `ward_energy_summary`, `health_advisories`, `dlq`) use 14–90 day retention appropriate to their downstream use.

B.2 Producers (bus_gps & air_quality)

- **`producers/bus_gps_producer.py`** — keys every message by `route_id` so all positions for a route land on one partition and stay **ordered**. It runs 8 routes × 3 buses and deliberately places two Route-R101 buses ~55 m apart so the Task C bunching detector has something to fire on.
- **`producers/air_quality_producer.py`** — implements **at-least-once** semantics: `acks=all` plus a synchronous `future.get()` confirmation and a manual retry loop (`MAX_SEND_RETRIES=3`) for the timeouts the brief describes. It injects the required **5% null-AQI** failures, which it logs and still forwards so the DLQ stage can catch them.

> **Why at-least-once here?** A missed AQI reading could mean a missed public-health emergency, so we prefer to risk a duplicate (cheap to de-duplicate downstream) over a lost reading. Ordering matters for bus GPS, hence the `route_id` key; throughput matters for AQI, hence retry-with-ack.

B.3 Priority Consumer Architecture

Two consumer groups read the same `traffic_signals` topic:

- **HIGH_PRIORITY** (`consumers/priority_consumer_high.py`) — a single consumer reading *all* partitions, feeding the real-time signal-control system.
- **STANDARD_PRIORITY** (`consumers/priority_consumer_standard.py`, run 3×) — three consumers sharing the partitions for analytics. A `--slow` flag adds 0.4 s/message to simulate a processing slowdown.

Because the two groups have independent offsets, slowing STANDARD does not affect HIGH. `consumers/lag_monitor.py` proves this by printing each group's lag from committed offsets vs end offsets: HIGH stays at near-zero lag while STANDARD's lag grows under load.

B.4 Real-Time Enrichment (Stream–Table Join)

`enrichment/bus_gps_enrichment.py` joins the live bus_gps stream with the static `data/route_schedule.csv` to add `scheduled_arrival_time`, `route_name`, and `terminal`, writing the result to `urbanpulse.bus_gps_enriched` — the foundation of the ETA service.

> **Note on Kafka Streams.** The Kafka Streams API is JVM-only and has no Python binding. To keep the project beginner-friendly and entirely in Python, this job implements the same **stream–KTable join pattern** faithfully: the CSV is loaded into an in-memory keyed table (the "KTable"), and each incoming GPS event is enriched by key lookup and re-emitted. The semantics match; only the runtime differs.

B.5 Dead-Letter Queue (DLQ)

`dlq/dlq_validator.py` validates air_quality and bus_gps events and routes failures to `urbanpulse.dlq` with an `error_reason` field:

- `NULL_AQI` — missing AQI value
- `AQI_OUT_OF_RANGE` — AQI outside 0–500
- `NULL_GPS` / `IMPOSSIBLE_GPS` — missing or out-of-bounds lat/lon

`dlq/dlq_report.py` consumes the DLQ over a configurable window (default 5 minutes), tallies the error-type distribution with a counter, prints it, and saves `output/dlq_report.txt` — the required 5-minute DLQ report.

Task C — Processing Layer

C.1 Flink Incident Detection (PyFlink DataStream)

Three detectors use keyed state and event-time watermarks and emit to `urbanpulse.incidents`. Shared helpers (Kafka connector JAR discovery, a haversine distance function, timestamp parsing) live in `flink/_flink_common.py`.

Detector	File	Logic
(a) AQI Emergency	`flink/aqi_emergency.py`	Filters air_quality for `AQI > 300` and emits an alert within the 2-minute SLA using an event-time source.
(b) Traffic Gridlock	`flink/traffic_gridlock.py`	Keyed by `junction_id`; a `KeyedProcessFunction` with `ValueState` counts consecutive cycles where `avg_wait_sec > 180` and fires after 3 in a row, reporting junction and zone.
(c) Bus Bunching	`flink/bus_bunching.py`	Keyed by `route_id`; `MapState` tracks per-bus positions and pair timers, firing when two buses stay within 200 m (haversine) for >5 min, with a cooldown to avoid alert spam.

C.2 Spark Structured Streaming (Ward Analytics)

- **`spark/ward_energy_summary.py`** — reads `urbanpulse.smart_meters`, applies a 5-minute watermark and a **15-minute tumbling window** per `ward_id`, and computes `total_kwh_consumed`, `avg_power_factor`, and `peak_voltage`. A `foreachBatch` sink writes simultaneously to the `urbanpulse.ward_energy_summary` topic and to Parquet **partitioned by `ward_id` and `date`** for historical trend analysis.
- **`spark/aqi_health_advisory.py`** — a Streaming SQL query that computes a **10-minute rolling average AQI per zone**, joins the static `data/zone_profile.csv` (zone name, population, schools), filters `rolling_avg_aqi > 150`, and writes enriched advisories to `urbanpulse.health_advisories` using **Update output mode**, exactly as specified.

C.3 Flink vs Spark — Which Engine for Which Job

The two jobs above need different engines, and the choice falls out of four factors.

Factor	Bus Bunching → Flink	Ward Energy → Spark
State size	Small but fine-grained: per-bus positions and per-pair timers, updated on every event. Flink keyed state fits exactly.	Bounded aggregates per ward per open window, updated once per micro-batch. Comfortable for Spark.
Latency requirement	Real-time, event-at-a-time; must fire the moment the 5-min threshold is crossed, correct under out-of-order GPS via watermarks.	Minute-scale; feeds councillor dashboards/reports. Micro-batches every few seconds are ample.
Recovery time objective	Fast: resumes per-route timers from keyed checkpoints rather than replaying from zero.	Relaxed: brief dashboard lag is fine, and reports can be recomputed deterministically from Parquet.
Operational complexity	Higher to learn, but the only engine that expresses "same key, sustained spatial condition, over event time" cleanly.	Lower: declarative windowed SQL with a `foreachBatch` sink; reuses existing analytics skills.

Conclusion. Bus bunching belongs on **Flink** — low-latency, event-time, fine-grained per-key state with timers. Ward energy aggregation belongs on **Spark** — windowed batch aggregation over relaxed latency with first-class partitioned Parquet for auditable reprocessing. This split is exactly why UrbanPulse adopts the Lambda architecture chosen in Task A.

On testing. All 22 Python modules pass `python -m py_compile`. The pipelines are designed to run on the beginner setup in `EXECUTION_GUIDE.md` against the Dockerised Kafka cluster; live end-to-end runs require Docker, a JDK, and the Flink/Spark Kafka connector JARs as that guide describes.